

AI-Assisted Course Management System

Final Group Project SPRING 2026

Sriram Madduri | Rakshitha Srinivasa | Ankush Rai

Department of Computer Science and Engineering

What is the project about?

An AI-assisted course management platform that supports course creation, assignment submission, automated analysis, grading, feedback generation, review workflows, notifications and traceability.

The system demonstrates 18 GoF design patterns across Creational, Structural and Behavioral categories

- **Instructors** — Step-by-step assignment building, rubric creation, and manual grading overrides.
- **Students** — Sandbox code execution tracking, iterative drafting and real-time diagnostic feedback.

BUILT WITH



Java 21



Spring Boot 3



Maven



JUnit 5



Basic HTML returned by
Spring controllers for the
landing and trace pages

What Problem are we solving?

- **Administrative Fatigue:** Manual evaluation of hundreds of complex code structures creates massive feedback delays and grading inconsistencies.
- **Fragile Third-Party Dependencies:** Direct, un-insulated API calls to AI services create unpredictable latency, high operating costs, and vulnerability to downtime.
- **Security and Execution Overhead:** Compiling and checking student-submitted Java code can expose infrastructure if not structurally isolated from validation rules.

Features Implemented

Hierarchical Course Management

Structured Course, Module, Assignment trees with automated template setup.

Multi-Stage Validation Pipeline

Sequential automated checks for file size, extensions, and submission deadlines.

Isolated Code Sandbox

Safe, deterministic Java execution and unit testing in a mock environment.

AI-Powered Feedback Engine

Specialized analysis for code and text with rubric-mapped critiques.

Intelligent Analysis Caching

Proxy layer to intercept repeated requests and eliminate redundant API costs.

Event-Driven Workflow

Real-time state tracking with asynchronous In-App notifications.

Creational Patterns Used, Where and Why?

Factory Method

SubmissionAnalyzerFactory

The system needs to analyze submissions, but the specific concrete analyzer (Code vs. Text) is decided at runtime. This avoids leaking implementation details into the submission workflow.

Abstract Factory

CourseContentFactory

Used to spawn families of related objects (Modules, Assignments, Rubrics) that must stay compatible. Ensures that a Project-Based course uses Project templates rather than Standard ones.

Builder

AssignmentBuilder

Assignments are complex objects with numerous optional fields. The builder handles step-by-step construction and enforces strict validation rules before the object is instantiated.

Structural Patterns Used, Where and Why?

Composite

CourseComponent

Treats the curriculum as a tree structure. This allows the system to manage an entire Module or a single Assignment leaf using the same unified interface.

Adapter

MockCodeSandboxAdapter

Wraps external mock sandbox APIs into internal domestic contracts. Isolates the core logic from signature changes in third-party or mock execution environments.

Facade

SubmissionWorkflowFacade

Simplifies complex orchestration for the controllers. Provides a high-level entry point that hides the interaction between validators, repositories, and state machines.

Decorator

TraceableFeedbackDecorator

Augments feedback results with additional metadata (timestamps, audit trails) dynamically without modifying the base feedback generator classes.

Proxy

CachedAnalysisServiceProxy

Acts as a gatekeeper for expensive AI services. Intercepts repeated submissions and returns cached results to significantly reduce API latency and cost.

Bridge

NotificationSender / Message

Decouples the message abstraction from the delivery implementation. Allows "In-App" and "Email" alerts to vary independently from the content being sent.

Behavioral Patterns Used, Where and Why?

Mediator

SubmissionWorkflowMediator

Centralizes communication between decoupled components (Validators, Analyzers, Repositories), ensuring they don't reference each other directly.

State

SubmissionState

Encapsulates state-specific behavior as the submission transitions from **Draft** to **Analyzing** and finally to **Finalized** status.

Strategy

GradingStrategy

Allows the runtime swapping of grading algorithms (e.g., Rubric-based vs. Code-Test results) without modifying core submission logic.

Chain of Responsibility

SubmissionValidationHandler

Sequences validation checks (Size, Type, Deadline) into independent, ordered links that can be easily added or re-ordered.

Template Method

AbstractSubmissionAnalyzer

Defines the fixed algorithmic skeleton for submission analysis, while allowing Code and Text subclasses to implement specific parsing details.

Observer

DomainEventPublisher

Broadcasts events (like **FeedbackFinalized**) to independent listeners (In-App, Email) without coupling the core feedback workflow.

Command

CourseCommand

Encapsulates instructor actions as executable objects, enabling the system to maintain a searchable history of course and assignment modifications.

Memento

FeedbackDraftMemento

Captures and restores the internal state of feedback drafts, allowing instructors to "undo" or "rollback" edits during the manual review process.

Iterator

CourseComponentIterator

Provides a standard way to traverse the complex course hierarchy (Modules and Assignments) without exposing the internal tree structure.

How It Works

End-to-End System Workflow

1. Setup & Entry

Courses structure as a tree (Composite). The Facade provides a clean entry for submissions, coordinated by the Mediator.



2. Validation

Submissions pass through a Chain of Responsibility. States move from Draft to Analyzing. A Proxy checks caching to save API costs.



3. Analysis

Sandbox execution (Adapter) validates code. A Template Method defines the grading flow, while Decorators enrich feedback metadata.



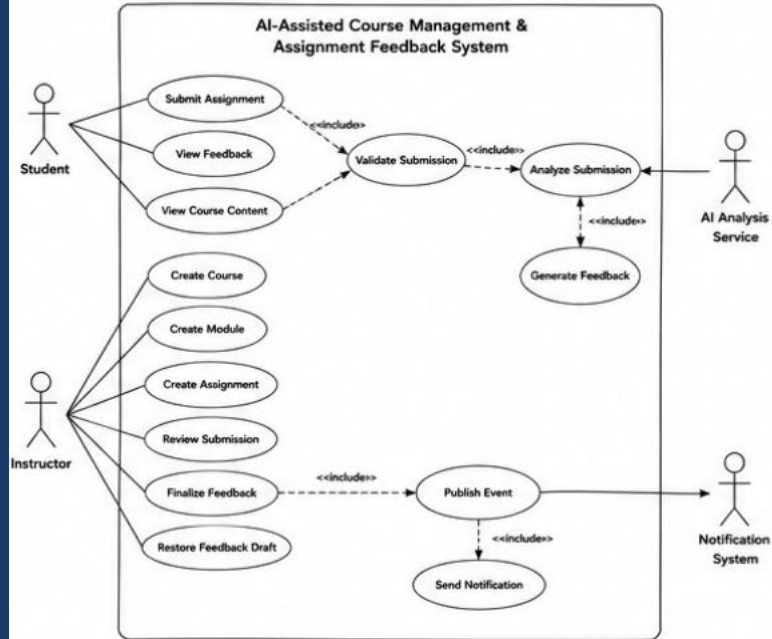
4. Delivery

Instructors use Memento to edit drafts. Observer publishes events, and Bridge delivers multi-channel notifications (App/Email).

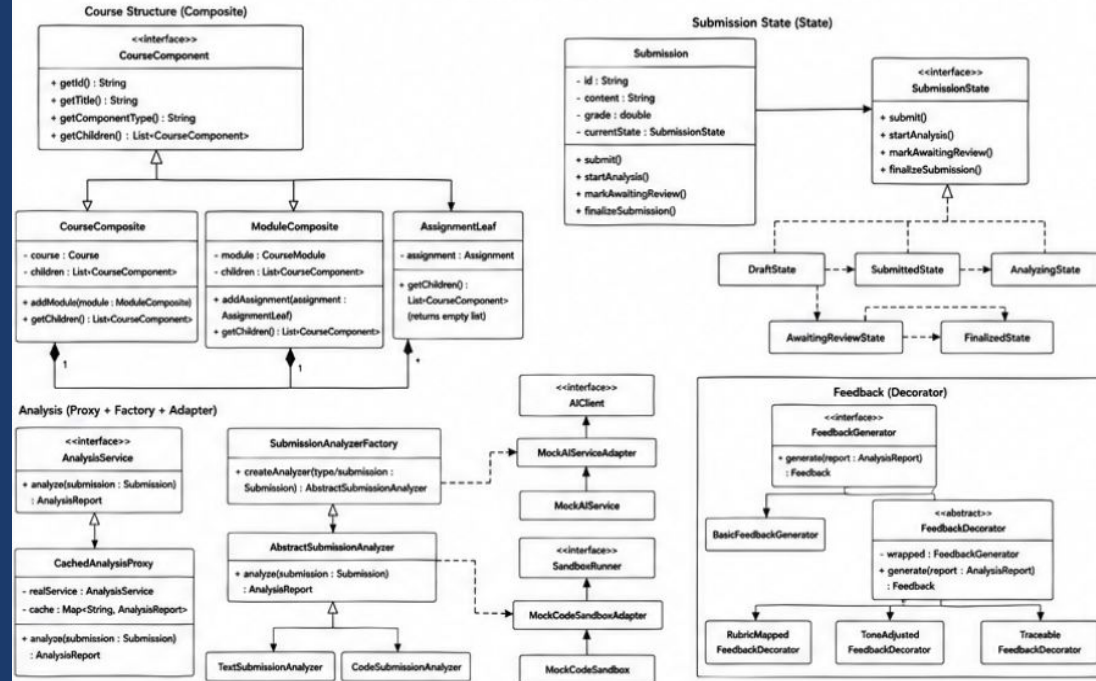
UML Diagrams

1. USE CASE DIAGRAM

AI-Assisted Course Management & Assignment Feedback System

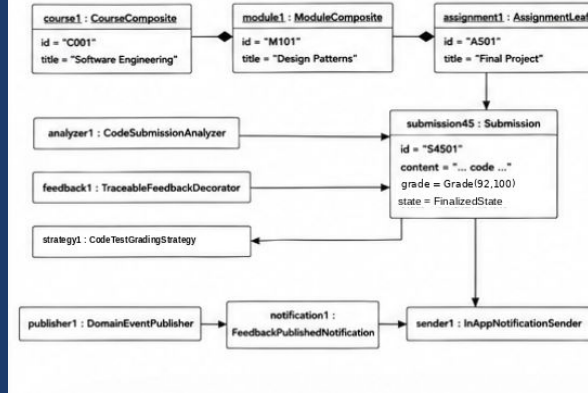


2. CLASS DIAGRAM

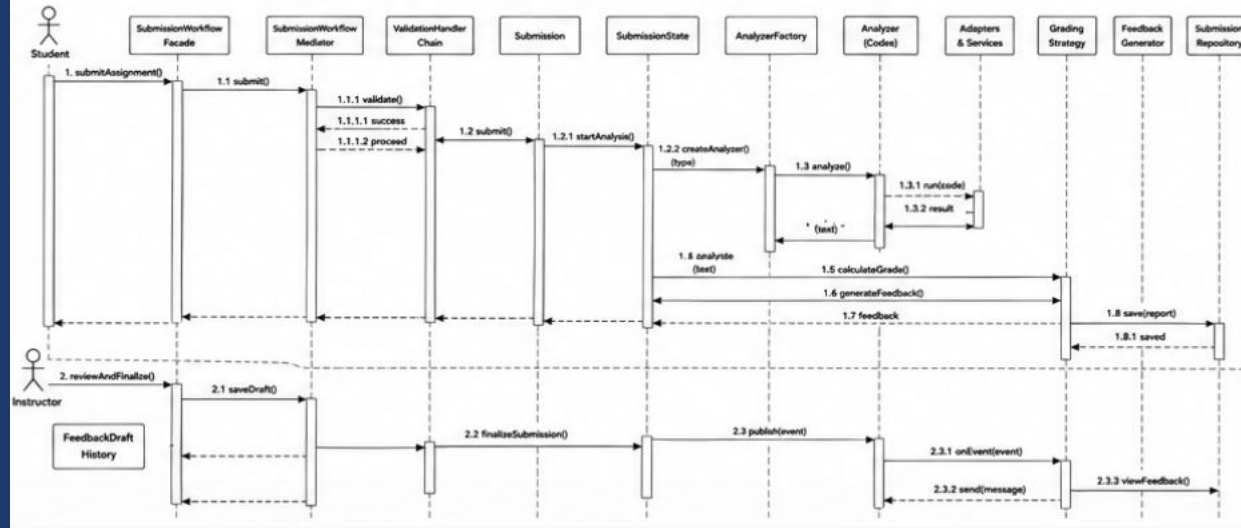


UML Diagrams

3. OBJECT DIAGRAM (Example Runtime Snapshot)



4. SEQUENCE DIAGRAM (End-to-End Submission & Feedback Lifecycle)





Demo





Santa Clara
UNIVERSITY

**Thank You
for your
Attention.**

**Any
Questions?**